Name: _____ **Points:** _____

S-Number: _____ **Date:** _____

1. General Information

- **Group size:** The project has to be done in groups of 4–5 students.
- **Hardware:** The project has to run on the provided Raspberry Pi 3.
- **Deadline:** The project has to be submitted on Moodle by 21.6. at 23:55.
- **Presentation date:** The project will be presented on 22.6. in front of both groups.

2. Project Description

The idea of the project is a rigged game of *rock, paper, scissors*. The Pi should play the game against a human player. In a fair game, the Pi would select a random gesture, wait until the player has decided on a gesture, and finally calculate if its gesture wins against the gesture of the player. But our game should be rigged. If the player is wearing a specific accessory the player should always win the game and if the player is not wearing the accessory the player should always lose the game.

You can choose the accessory freely. It can be something similar to a *wristband or ring*. What is important though is that only this specific accessory should enable winning against the Pi, other wristbands/rings should not help to achieve wins. The game should function as intended with hands that are physically different (e.g. different skin tones).Therefore all the members of a group should contribute to the creation of the dataset.

The Pi displays its choice (e.g., *rock*) as a symbol on the LED matrix for two seconds. After this symbol, the entire screen should turn green if you won or red if you lost. You can use the exercise files to write to the LED matrix and read the camera data.

Finally, an important hint: Make sure that your implementation does not confuse the gestures before unveiling your element with an actual game gesture (e.g. shacking your fist should not be classified as rock). Your implementation can involve detecting a stable gesture without movement before classification or you can implement a countdown on the LED matrix to ensure that the player presents an interpretable gesture at the correct time.

Figure 1. Human playing against Raspberry Pi.

3. Technical Specification

- The executable running on the Pi has to be implemented entirely in C++.
- The executable has to be started by a service after the Pi boots (no matter if the Pi is connected to the internet or not).
- The TFLite files for the model(s) have to stay within a memory budget of 100MB in total.
- The application has to run with at least 30FPS, that means at least 30 inference runs per second.

4. Presentation

- The presentation must include a breakdown of your application and model architectures (this includes the dataflow through the application).
- The presentation must include the numbers for inference runs per second and TFLite file sizes.
- The presentation must include accuracy numbers during training/test and how well this accuracy translates to real world performance.
- The presentation must include an overview on what issues and pitfalls you encountered during the implementation.
- The presentation must include a live demonstration with both a person that is included in the dataset as well as a person that was not in the dataset (with and without the accessory).

5. Moodle Upload Requirements

The Moodle upload must contain all files required to build, run, and evaluate the project on the provided Raspberry Pi 3 hardware.

- The complete C++ source code of the application.
- The trained TFLite model file(s) used by the application and the Python scripts for training and converting the models.
- All additional files required to run the application on the Raspberry Pi, including configuration files, service files, scripts, etc.
- A short description explaining how to install, start and run the application on the Raspberry Pi.
- The presentation slides.